

第 6 讲：应用题（下）直播内容

应用题按主考点统计

专题	主考次数	说明
线性表	0	2009~2026 没有一题以线性表应用为主
栈、队列和数组	3	2011 数组（矩阵压缩存储）；2019 队列；2026 栈
树与二叉树	3	2012 哈夫曼树&归并排序交叉、2016 正则树、2020 前缀编码哈夫曼树、2022 堆；
图	7	历史考频最高，且多次结合计算机网络、工程调度等现实场景
查找	3	2010、2013、2024；其中两次为散列表
排序	3	2012、2021、2023；2022 堆&排序交叉

分专题训练（预测题）

线性表的应用

稀疏多项式存储

树的存储结构

树的存储结构

对比：树的孩子表示法、图的邻接表、拉链法处理冲突的散列表

并查集及其应用

树——并查集与动态连通性

稀疏多项式存储

在计算机代数系统中，公式化简、符号求导和方程求解等操作都需要频繁进行多项式运算。某系统需要存储整系数多项式，多项式的最高指数可达999999999，但非零项很少。

现有

$$P(x) = 5x^{12} - 3x^7 + 6x^5 + 2x^3 + 1$$

$$Q(x) = -5x^{12} + 4x^9 + 3x^7 - 2x^5 - 2x^3 - 2x$$

需要计算

$$R(x) = P(x) + Q(x)$$

- (1) 应选择顺序表还是单链表存储多项式？说明理由。(2分)
- (2) 用C或C++语言给出数据结点的定义。(2分)
- (3) 画出P、Q的初始存储结构示意图。(2分)
- (4) 求R(x)，并画出其存储结构示意图。(2分)
- (5) 简述多项式加法的实现过程，并分析其时间复杂度和辅助空间复杂度。设P、Q的非零项数分别为m、n。(2分)

参考答案

(1) 存储结构

应选择单链表。多项式的最高指数很大，但非零项很少，若按指数建立顺序表会浪费大量空间。单链表只存储非零项，也便于插入、删除和合并。

(2) 数据结点定义

```
typedef struct PolyNode {  
    int coef;           // 系数  
    int exp;            // 指数  
    struct PolyNode *next;  
} PolyNode, *Polynomial;
```

(3) P、Q的初始存储结构

结点中的两个数依次表示“系数、指数”。

```
P  
|  
▼  
头结点 → [5,12] → [-3,7] → [6,5] → [2,3] → [1,0] → NULL  
  
Q
```

↓
头节点 → [-5, 12] → [4, 9] → [3, 7] → [-2, 5] → [-2, 3] → [-2, 1] → NULL

(4) 结果多项式及其存储结构

$$R(x) = 4x^9 + 4x^5 - 2x + 1$$

一种可能的结点复用结果为：

R
↓
头节点 → [4, 9] → [4, 5] → [-2, 1] → [1, 0] → NULL

其中，指数为12、7、3的同指数项相加后系数为0，不保留相应结点。

(5) 合并过程与复杂度

为 $R(x)$ 新建一个头结点，作为一条新链表，并用一个尾指针指向该链表的最后一个结点（初始时指向头结点），后续处理的结点均接到链尾。

每次分别取 P、Q 当前指数最大的结点（即头结点后面的第一个结点），比较这两个结点的指数：

- 若某一方的指数更大，则把该结点直接接到结果链表 R 的尾部，并让这一方的指针后移，指向其下一个（次大指数）结点；
- 若两个结点指数相同，则将系数相加：若加和不为 0 则保留其中一个结点（更新系数）接入结果链表 R 的尾部，并删除另一个结点；若加和为 0 则两个结点都删除。

如此不断取 P、Q 中“当前指数最大的结点”进行处理，直到某条链表被处理完；再把另一条链表尚未处理的剩余部分整体接到结果表尾。

时间复杂度为 $O(m+n)$ 。空间复杂度为 $O(1)$ 。

树的存储结构

双亲表示法（并查集原理雷同）

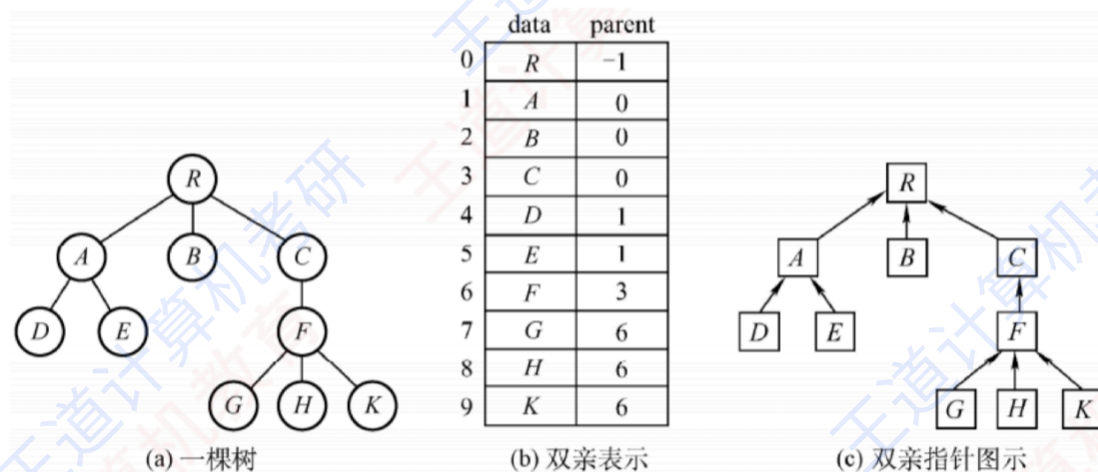
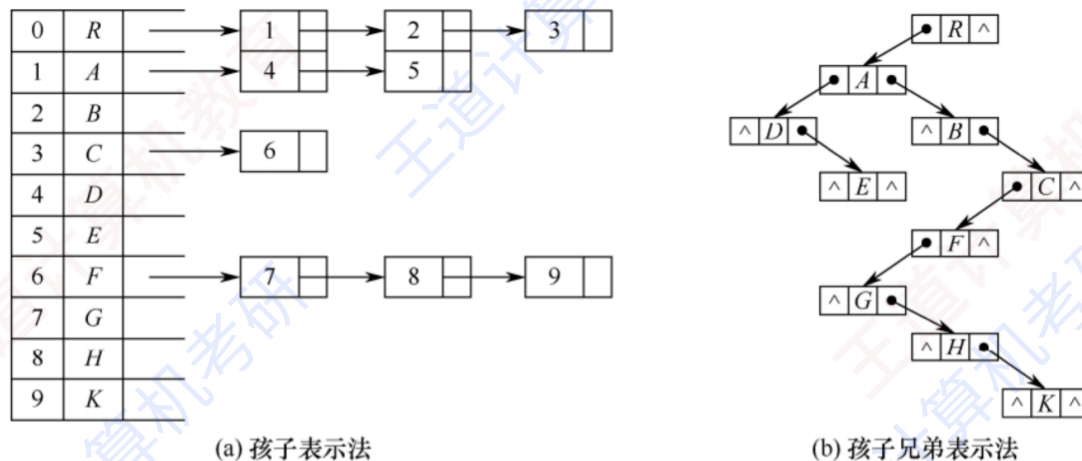


图 5.20 树的双亲表示法

孩子表示法、孩子兄弟表示法



三种存储结构定义总结

一、双亲表示法

思想：用一段连续空间（顺序表）存储各结点，每个结点在保存数据的同时，记录其双亲在数组中的下标。根结点无双亲，其双亲域约定为 -1。

```
#define MAX_TREE_SIZE 100
typedef struct {
```

```

ElemType data;          // 结点数据
int parent;             // 双亲在数组中的下标 (根为 -1)
} PTNode;
typedef struct {
    PTNode nodes[MAX_TREE_SIZE];
    int n;               // 结点总数
} PTree;

```

- **优点：**求某结点的双亲很快，时间复杂度 $O(1)$ 。
- **缺点：**求孩子需遍历整个数组；求兄弟同样不便。
- **适用：**与并查集原理雷同，常用于频繁“找双亲/找根”、判断连通性的场景。

二、孩子表示法

思想：把每个结点的所有孩子用一条**单链表**（孩子链表）串起来；再用一个顺序表存放各结点的数据以及指向其孩子链表的头指针。

```

typedef struct CNode {    // 孩子链表结点
    int child;           // 该孩子在数组中的下标
    struct CNode *next;  // 指向下一个孩子
} CNode;
typedef struct {          // 顺序表中的表头结点
    ElemType data;
    struct CNode *firstChild; // 指向第一个孩子的指针
} CBox;
typedef struct {
    CBox nodes[MAX_TREE_SIZE];
    int n, r;             // 结点总数、根在数组中的位置 (根也可固定在 0 号位置)
} CTree;

```

- **优点：**求某结点的孩子非常方便（顺着孩子链表走即可）。
- **缺点：**求双亲需要遍历所有孩子链表。

三、孩子兄弟表示法（二叉树表示法）

思想：给每个结点设两个指针——`firstchild` 指向它的**第一个孩子**，`nextsibling` 指向它的**下一个兄弟**。这样任意一棵树（或森林）都被唯一地转化成一棵**二叉树**。

```

typedef struct CSNode {
    ElemType data;
    struct CSNode *firstchild, *nextsibling; // 第一个孩子、下一个兄弟
} CSNode, *CSTree;

```

- **优点：**存储灵活、便于把树/森林与二叉树相互转换，是三者中最常用的表示法。
- **缺点：**从孩子找双亲不方便（如有需要，可再增设一个 `parent` 指针）。

孩子兄弟表示法的遍历 —— 与二叉树递归遍历对比

孩子兄弟表示法本质上就是一棵二叉树，只要做如下“换名”，二叉树的一切递归套路都能直接搬过来：

二叉树	孩子兄弟表示法
左孩子 <code>lchild</code>	第一个孩子 <code>firstchild</code>
右孩子 <code>rchild</code>	下一个兄弟 <code>nextsibling</code>

二叉树的先序遍历：

```
void PreOrder(BiTree T) {
    if (T != NULL) {
        visit(T);
        PreOrder(T->lchild); // 递归左子树
        PreOrder(T->rchild); // 递归右子树
    }
}
```

孩子兄弟表示法“树的先根遍历”：

```
void PreOrderTree(CSTree T) {
    if (T != NULL) {
        visit(T);
        PreOrderTree(T->firstchild); // 相当于二叉树的左子树
        PreOrderTree(T->nextsibling); // 相当于二叉树的右子树
    }
}
```

二叉树的中序遍历：

```
void InOrder(BiTree T) {
    if (T != NULL) {
        InOrder(T->lchild); // 递归左子树
        visit(T);
        InOrder(T->rchild); // 递归右子树
    }
}
```

孩子兄弟表示法“树的后根遍历”：

```
void PostOrderTree(CSTree T) {
    if (T != NULL) {
        PostOrderTree(T->firstchild); // 相当于二叉树的左子树
        visit(T);
        PostOrderTree(T->nextsibling); // 相当于二叉树的右子树
    }
}
```

两段代码结构完全一致，只是把 `lchild/rchild` 换成了 `firstchild/nextsibling`。因此不必额外记忆一套“树的遍历”算法，套用二叉树的写法即可。

重要结论（预防算法题考“树”）

若一棵树用孩子兄弟表示法存储，则：

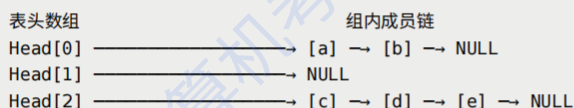
- 树的先根遍历序列 = 其对应二叉树的先序遍历序列；
- 树的后根遍历序列 = 其对应二叉树的中序遍历序列。

注意：树没有“中根遍历”的概念（结点孩子数不定），所以只与二叉树的先序、中序两种建立对应关系。

对比：树的孩子表示法、图的邻接表、拉链法处理冲突的散列表

Key：这四种结构都可以看成“定长表头（数组）+ 变长成员链”

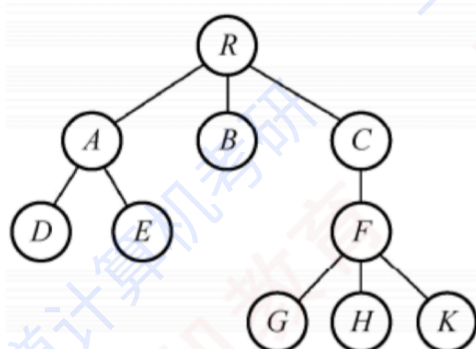
一套骨架，四种含义



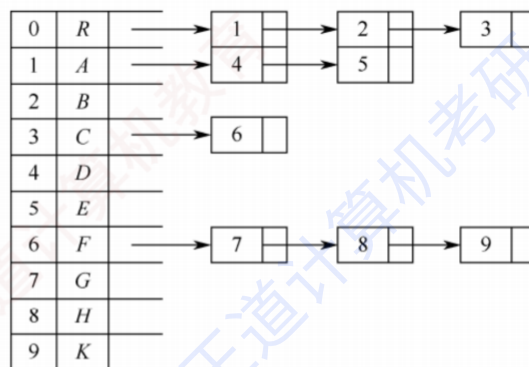
结构	表头下标表示	链结点表示	进入同一链的原因
树的孩子表示法	某个树结点	该结点的一个孩子	双亲相同
图的邻接表	某个图顶点	一个邻接点或一条边	关联于同一顶点
拉链法散列表	某个散列地址（桶号）	一个关键字或记录	散列地址相同
基数排序辅助结构	当前位的某个数字值	一个待排序记录	当前位数字相同

1. 树的孩子表示法

i 号表项对应结点 i ；其链表就是该结点的“孩子名单”。



(a) 一棵树

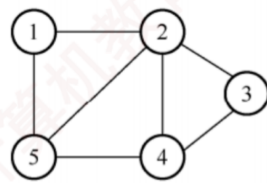


(a) 孩子表示法

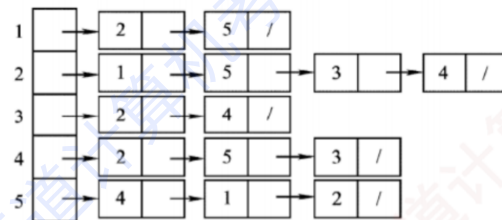
表头项通常保存 结点数据 和 第一孩子链节点指针；孩子链结点保存 孩子的编号 及 next 指针。

2. 图的邻接表

i 号表项对应顶点 v_i ；其链表列出与 v_i 邻接的顶点或关联的边。

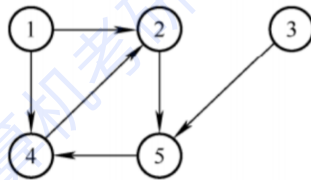


(a) 无向图 G

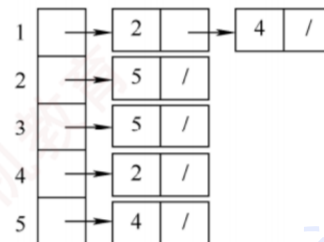


(b) 无向图 G 的邻接表的表示

图 6.7 无向图的邻接表表示



(a) 有向图 G



(b) 有向图 G 的邻接表的表示

图 6.8 有向图的邻接表表示

表头项通常保存 顶点信息 和 第一个边节点指针；边结点保存 邻接点编号 及 next 指针，带权图还可保存 权值 等边信息。

3. 拉链法处理冲突的散列表

i 号表项对应散列地址 i ；其链表保存所有满足 $H(key)=i$ 的记录。
散列表表长为 m 时设置 m 个桶；每条记录根据散列函数进入唯一的桶。

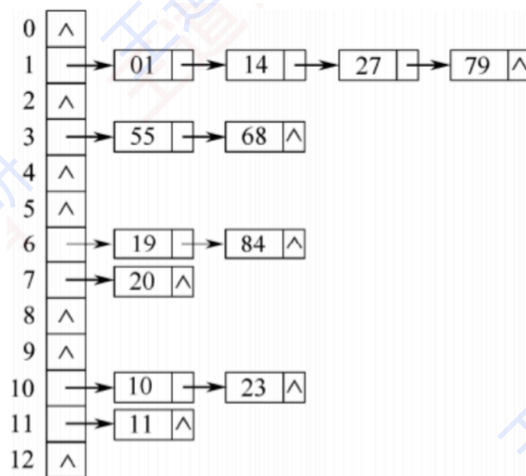


图 7.33 拉链法处理冲突的散列表

表头项（桶）通常保存该散列地址的 第一个记录结点指针（有的实现还会附带桶内 结点数 等辅助信息）；冲突链结点保存 关键字（或整条记录）及 next 指针

三种结构可以统一抽象为：

表头数组 + 若干变长成员链表

表头项 head[i]



通用数据结构定义框架

```
/* 链结点 */
typedef struct Node {
    存链节点信息：树存“孩子编号”；图存“邻接点编号”；散列表存“关键字或记录”
    struct Node *next;
} Node;

/* 表头项 */
typedef struct {
    存表头信息：树存“结点信息”，图存“顶点信息”，散列表可以什么都不存。
    Node *first;           // 第一个成员结点
} Head;

//定义表头数组。其中，NUM 对应树的节点数、图的顶点数、散列表的散列地址数
Head head[NUM];

//若“树、图”的结点/顶点数量不固定，可能发生变化，则可按如下定义：
typedef struct {
    Head head[MAX_HEAD_NUM];
    int headNum;           // 实际结点/顶点数
} TREE 或 Graph;
```

树——并查集与动态连通性

题目 (10 分)

某实验室有编号 0~7 的 8 台设备，初始时彼此不连通。系统依次收到以下连接请求：

```
(0,1), (1,2), (3,4), (2,4), (1,4),  
(5,6), (6,7), (4,7), (0,7)
```

若请求的两个端点已经连通，则该请求被记为“冗余连接”；否则合并两个连通分量。系统采用按规模合并并带路径压缩的并查集。

1. 按顺序处理请求，指出哪些请求是冗余连接。(4 分)
2. 处理完前 6 个请求后，各连通分量分别是什么？全部请求处理完后有几个连通分量？(2 分)
3. 说明并查集如何用于 Kruskal 算法判断一条候选边是否会形成环。(2 分)

参考答案与给分点

1. 处理到 (1,4) 时，1 和 4 已经通过 1-2-4 连通，因此它冗余；处理到 (0,7) 时，所有设备已经连通，因此它也冗余。其余 7 个请求均用于合并不同分量。(每个冗余请求及理由各 2 分)
2. 前 6 个请求后：

```
{0,1,2,3,4}, {5,6}, {7}
```

全部处理后只有 1 个连通分量。(2 分)

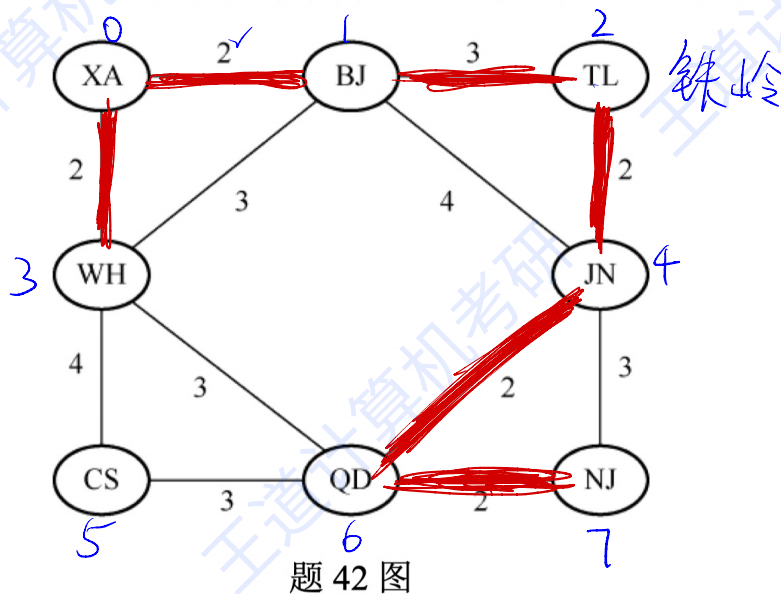
3. Kruskal 按边权从小到大考察边 (u,v) ：若 $\text{find}(u) == \text{find}(v)$ ，加入该边会在当前森林中形成环，应跳过；否则加入该边并执行 $\text{union}(u,v)$ 。(2 分)

一题多变

- 与 Kruskal 合并考察，要求同时给出 MST 边集与并查集状态。
- 把设备连接改成社交网络、岛屿连通、账户合并或网络故障恢复。

42. (12分) 拟建设一个光通信骨干网络连通 BJ、CS、XA、QD、JN、NJ、TL 和 WH 个城市，题 42 图中无向边上的权值表示两个城市间备选光纤的铺设费用。

8



题 42 图

克 → 选边

(0, 1) 2	✓ 0	2
(0, 2) 2	✓ 1	0
(2, 4) 2	2	-7
(4, 6) 2	3	0
(6, 7) 2	4	2
(1, 2) 3	5	-1
(1, 3) 3	X	
(3, 6) 3	6	2
(4, 7) 3	7	2
(5, 6) 3		
(1, 4) 4		
(3, 5) 4		

请回答下列问题。

(1) 仅从铺设费用角度出发，给出所有可能的最经济的光纤铺设方案（用带权图表示），并计算相应方案的总费用。

③手算分析算法运行过程/结果。

(2) 题 42 图可采用图的哪种存储结构？给出求解问题 (1) 所使用的算法名称。

④数据结构、算法的选择。

(3) 假设每个城市采用一个路由器按 (1) 中得到的最经济方案组网，主机 H1 直接连接在 TL 的路由器上，主机 H2 直接连接在 BJ 的路由器上。若 H1 向 H2 发送一个 TTL=5 的 IP 分组，则 H2 是否可以收到该 IP 分组？

③手算分析算法运行过程/结果。

堆排序 $\Rightarrow$ 小根

42. (10 分) 现有 n ($n > 100\,000$) 个数保存在一维数组 M 中，需要查找 M 中最小的 10 个数。请回答下列问题。

low 建堆 $\Rightarrow$

(1) 设计一个完成上述查找任务的算法，要求平均情况下的比较次数尽可能少，简

述其算法思想（不需要程序实现）。④数据结构、算法的选择。

⑥文字描述算法思想/过程。

(2) 说明你所设计的算法平均情况下的时间复杂度和空间复杂度。⑤算法性质分析

10元素堆 (大根堆)

int heap[10] ✓堆的应用 $\Rightarrow$ 找 Top-K

堆顶对比

OS
优先级调度 $\Rightarrow$ low 进程并发 $\rightarrow$ 辅助数组长度 K . 参见咸鱼 html

注：考点“堆排序”在 2022 年之前，仅被归类为“排序”；

2022 大纲更新后，“堆”也被归类于“树与二叉树” $\rightarrow$ 树与二叉树的应用 $\rightarrow$ 堆及其应用 $\rightarrow$ [] $\rightarrow$ [] $\rightarrow$ [] $\rightarrow$ []

10000 $\Rightarrow \log_2 10000 \Rightarrow 14$ 树高
2021年真题

注：2021 年真题并没有在

应用题

大根堆

42. (8 分) 已知某排序算法如下:

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count = (int *) malloc(sizeof(int) * n); //C++语言: count = new int[n];
    for(i=0; i<n; i++) count[i] = 0;
    for(i=0; i<n-1; i++)
        for(j=i+1; j<n; j++)
            if(a[i] < a[j]) count[j]++;
            else count[i]++;
    for(i=0; i<n; i++) b[count[i]] = a[i];
    free(count); //C++语言: delete count;
}
```

请回答下列问题。

1) 若有 $\text{int } a[] = \{25, -10, 25, 10, 11, 19\}, b[6];$, 则调用 $\text{cmpCountSort}(a, b, 6)$

后数组 b 中的内容是什么? ③手算分析算法运行过程/结果。

2) 若 a 中含有 n 个元素, 则算法执行过程中, 元素之间的比较次数是多少? ③手算分析算法运行过程/结果

3) 该算法是稳定的吗? 若是, 则阐述理由; 否则, 修改为稳定排序算法。⑤算法性质分析。